Connectionist Temporal Classification (CTC) with application to Optical Character Recognition (OCR)

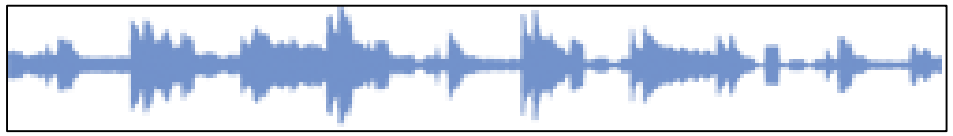
Siyang Wang

Outline

- Two long-standing tasks
  - speech recognition and OCR
- Motivation: Pre-CTC Methods
  - HMM
  - HMM-RNN hybrid
- Connectionist Temporal Classification(CTC)
- Applying CTC to OCR
- Disadvantages of CTC

Two long-standing tasks

- Speech recognition



→ “Hello world”

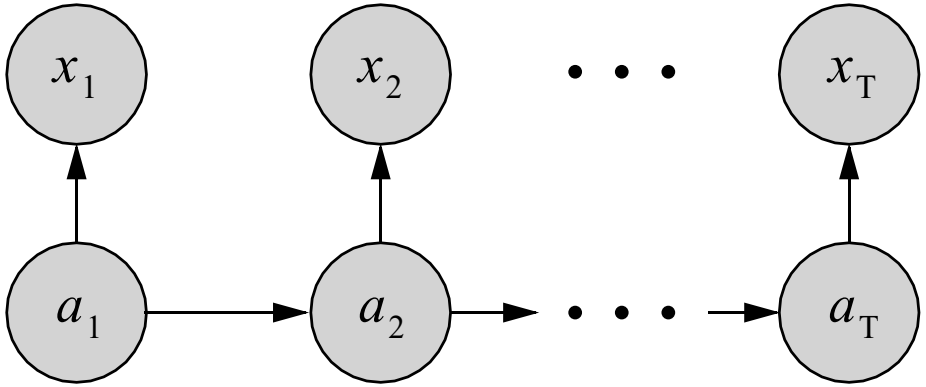
- Optical character recognition (OCR)

→ “Hello world”

A major difficulty

- No temporal correspondence (discussion question posted earlier)
  - Example: which segment of a sound signal sequence corresponds to a phoneme?
  - Ordering as a limited prior: not enough to easily establish correspondence
- Segmentation and alignment problems
  - Ambiguity: two connected phenom
  - Lack of per-frame labeling (difficult to obtain such labeling, also does not make much sense to do so)

Pre-CTC: Hidden Markov Models (HMM)



$x_t = observed\ state\ at\ t(sound\ signal)$

$a_t = hidden\ state\ at\ t\ (phoneme)$

https://distill.pub/2017/ctc/

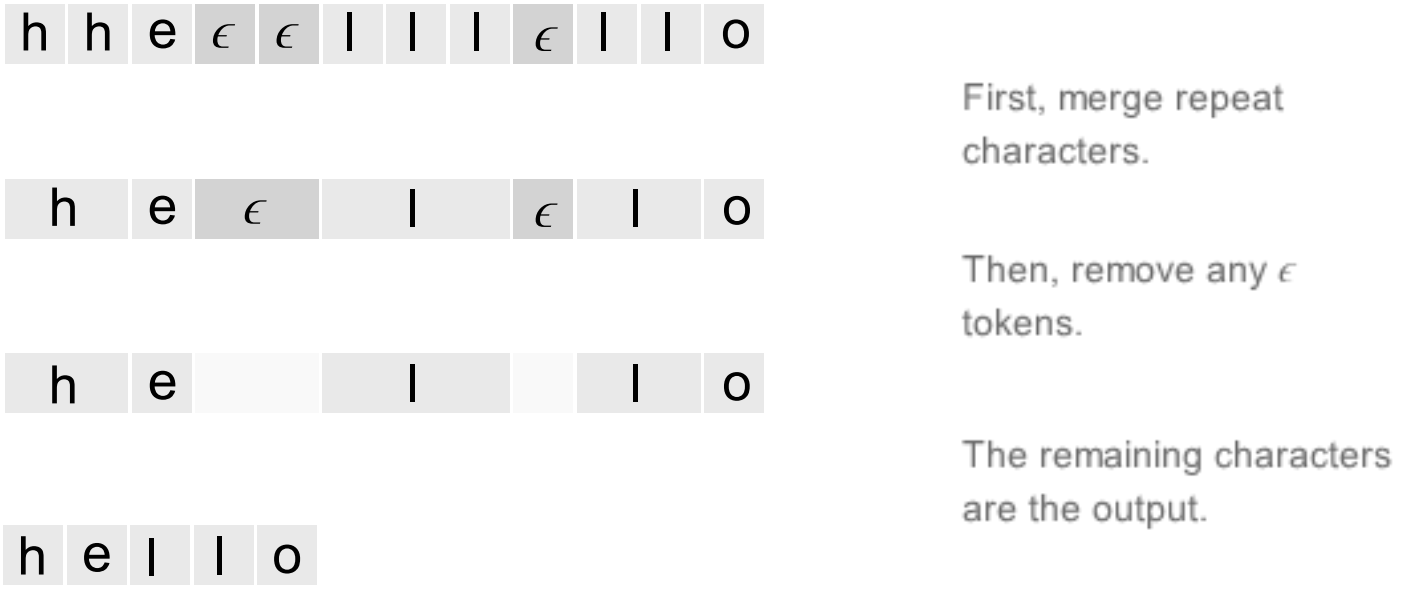
- Conditional Independence assumptions:
  - $P(x_t|a_1, \dots, a_T, x_1, \dots, x_T) = P(x_t|a_t)$
  - $P(a_t|a_1, \dots, a_T, x_1, \dots, x_T) = P(a_t|a_{t-1}) = P(a_{t'}|a_{t'-1})$
- Inference: Forward-backward (Viterbi's) Algorithm
- Training: EM Algorithm
- Simple segmentation strategy: combine connected hidden states to output predicted sequence

HMM Disadvantages(Graves, 2006)

- Inherently Generative (limits classification ability)
- Only limited RNN incorporation (identify local phenomes)
  - HMM-RNN hybrids
  - Does not allow applying RNN end-to-end
- However, more work has shown since CTC paper(2006):
  - Combining deep neural network (not necessarily RNN) to HMM performs well
  - Transducer in speech recognition (next lecture's presentation!)

Connectionist Temporal Classification (CTC)

- Alignment free transformation
  - Add a “blank” token to the pool of output classes/tokens
  - Consecutive same tokens between “blank” tokens are taken as one token
- Example:



https://distill.pub/2017/ctc/

How does this framework help classification?

- Define the classification problem: $X \rightarrow Y$
- But, both X and Y can vary in length in the same problem
- We want $P(Y|X)$ to MLE and back-prop

$$p(Y \mid X) = \sum_{A \in \mathcal{A}_{X,Y}} \prod_{t=1}^{T} p_t(a_t \mid X)$$

The CTC conditional **probability** **marginalizes** over the set of valid alignments computing the **probability** for a single alignment step-by-step.

https://distill.pub/2017/ctc/

CTC P(Y|X) example

$$p(Y \mid X) = \sum_{A \in \mathcal{A}_{X,Y}} \prod_{t=1}^{T} p_t(a_t \mid X)$$

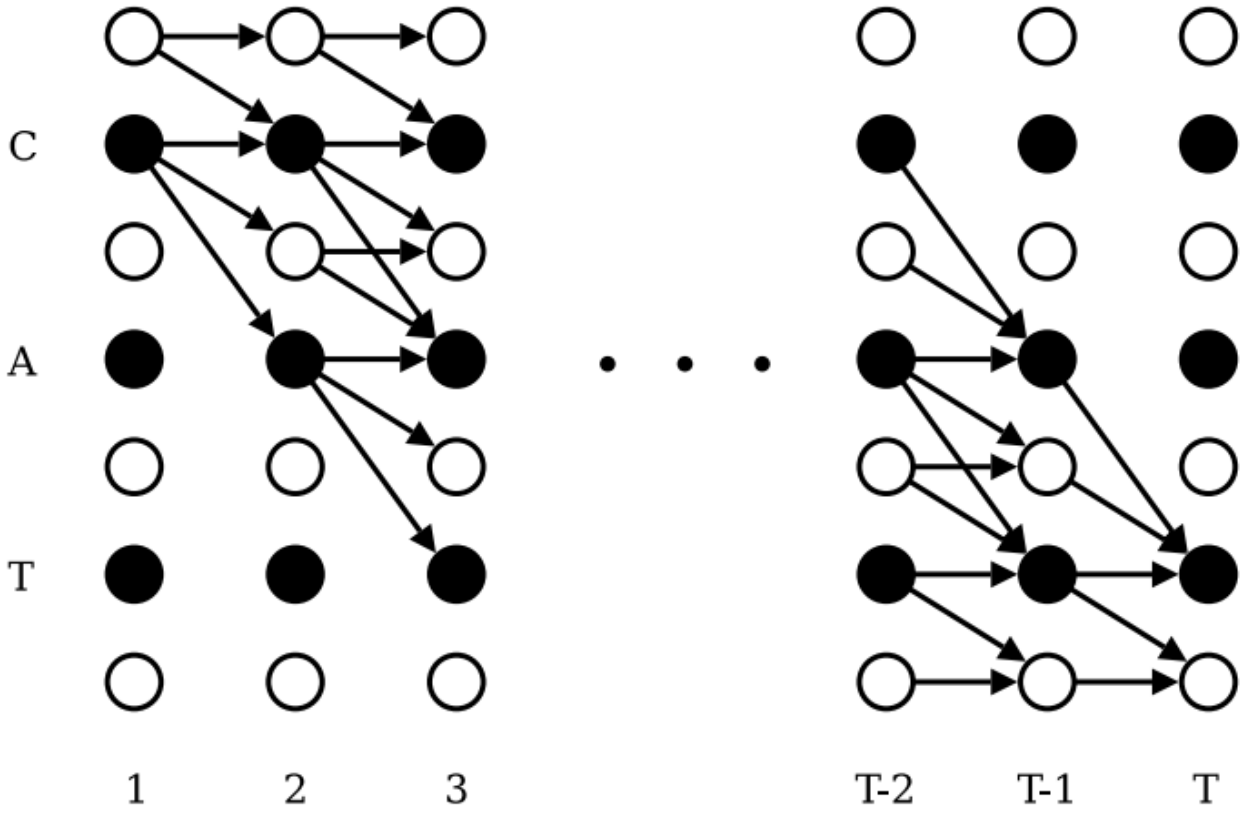
The CTC conditional **probability** **marginalizes** over the set of valid alignments computing the **probability** for a single alignment step-by-step.

https://distill.pub/2017/ctc/

t	1	2	3	4
P(“a”|X)	0.9	0.7	0.2	0.0
P(“m”|X)	0.1	0.2	0.0	0.9
P(“blank”|X)	0.0	0.1	0.8	0.1

$P(Y = "am"|X)$?

Efficient loss calculation: forward and backward algorithm (dynamic programming)



$$\alpha_t(s) \stackrel{\text{def}}{=} \sum_{\substack{\pi \in N^T: \\ \mathcal{B}(\pi_{1:t}) = \mathbf{l}_{1:s}}} \prod_{t'=1}^{t} y^{t'}_{\pi_{t'}}$$

$$\alpha_1(1) = y^1_b$$
$$\alpha_1(2) = y^1_{\mathbf{l}_1}$$
$$\alpha_1(s) = 0, \ \forall s > 2$$

and recursion

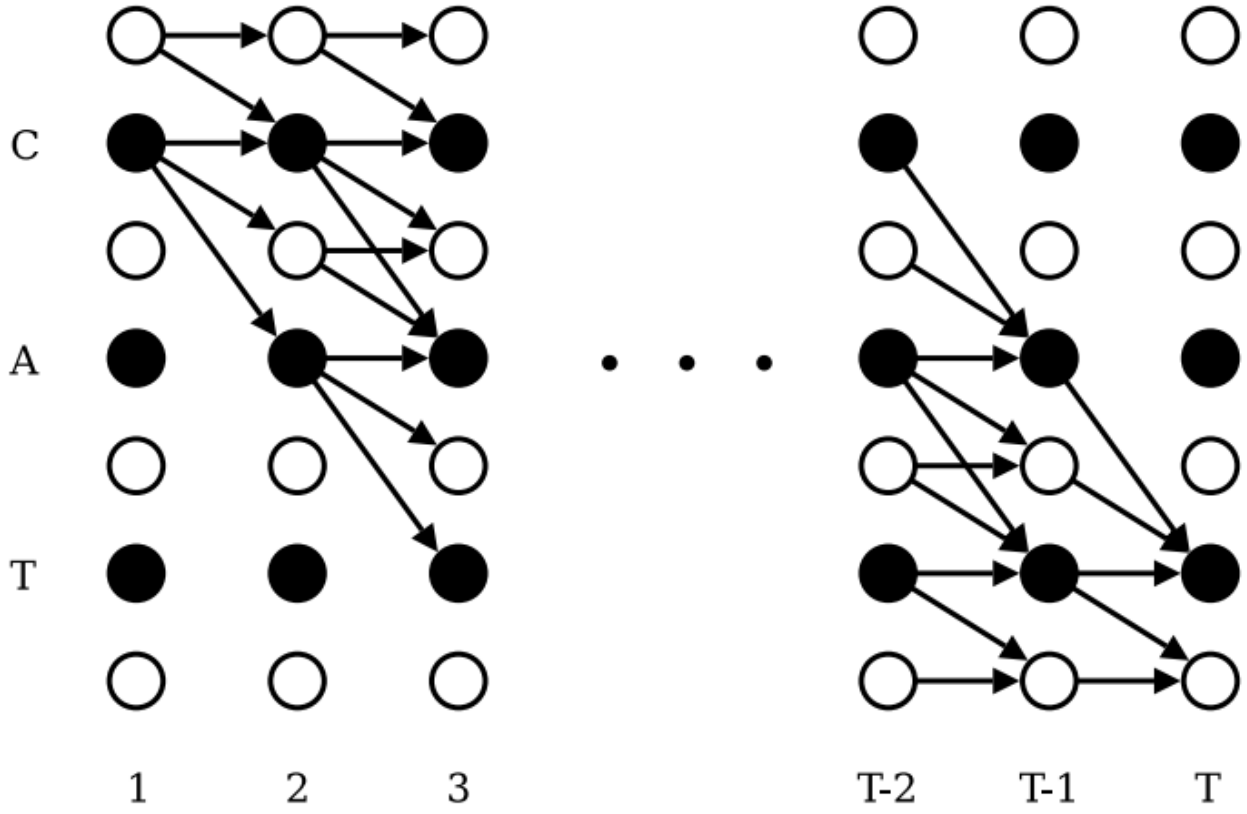
$$\alpha_t(s) = \begin{cases} \bar{\alpha}_t(s) y^t_{\mathbf{l}'_s} & \text{if } \mathbf{l}'_s = b \text{ or } \mathbf{l}'_{s-2} = \mathbf{l}'_s \\ \big(\bar{\alpha}_t(s) + \alpha_{t-1}(s-2)\big) y^t_{\mathbf{l}'_s} & \text{otherwise} \end{cases}$$

where

$$\bar{\alpha}_t(s) \stackrel{\text{def}}{=} \alpha_{t-1}(s) + \alpha_{t-1}(s-1).$$

ftp://ftp.idsia.ch/pub/juergen/icml2006.pdf

Forward pass case 1:

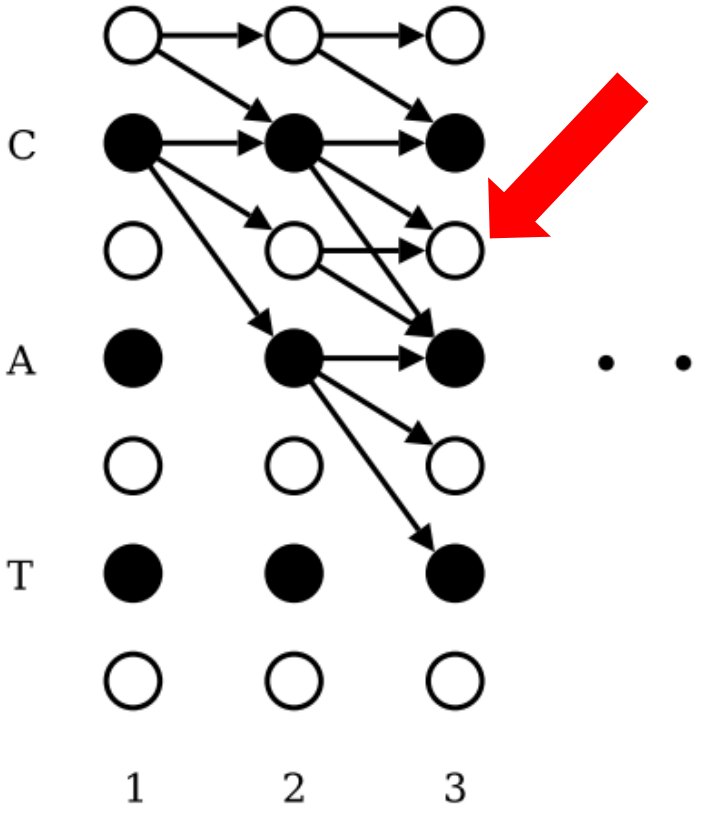


$$\alpha_t(s) = \begin{cases} \bar{\alpha}_t(s) y^t_{\mathbf{l}'_s} & \text{if } \mathbf{l}'_s = b \text{ or } \mathbf{l}'_{s-2} = \mathbf{l}'_s \\ \big(\bar{\alpha}_t(s) + \alpha_{t-1}(s-2)\big) y^t_{\mathbf{l}'_s} & \text{otherwise} \end{cases}$$

where

$$\bar{\alpha}_t(s) \overset{\text{def}}{=} \alpha_{t-1}(s) + \alpha_{t-1}(s-1).$$

Forward pass case 1A:

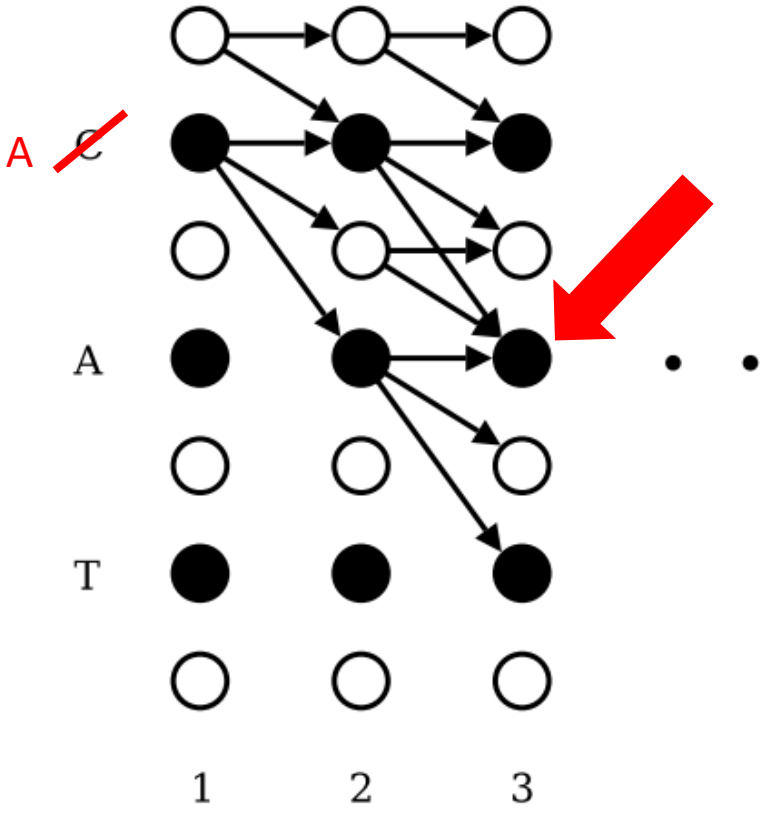


$$\alpha_t(s) = \begin{cases} \bar{\alpha}_t(s) y^t_{\mathbf{l}'_s} & \text{if } \mathbf{l}'_s = b \text{ or } \mathbf{l}'_{s-2} = \mathbf{l}'_s \\ \left(\bar{\alpha}_t(s) + \alpha_{t-1}(s-2)\right) y^t_{\mathbf{l}'_s} & \text{otherwise} \end{cases}$$

where

$$\bar{\alpha}_t(s) \overset{\text{def}}{=} \alpha_{t-1}(s) + \alpha_{t-1}(s-1).$$

Forward pass case 1B:

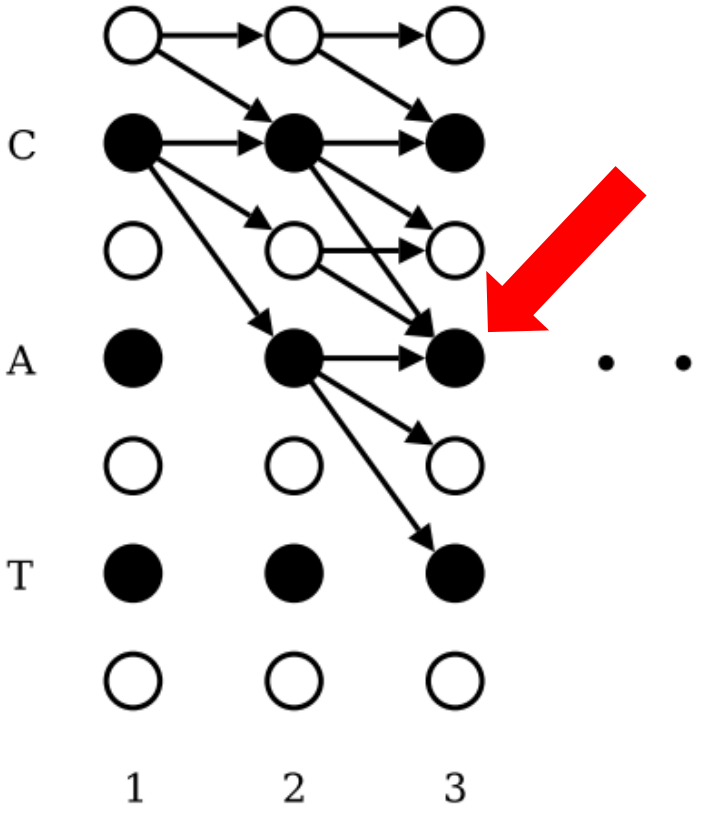


$$\alpha_t(s) = \begin{cases} \bar{\alpha}_t(s) y^t_{\mathbf{l}'_s} & \text{if } \mathbf{l}'_s = b \text{ or } \mathbf{l}'_{s-2} = \mathbf{l}'_s \\ \big(\bar{\alpha}_t(s) + \alpha_{t-1}(s-2)\big) y^t_{\mathbf{l}'_s} & \text{otherwise} \end{cases}$$

where

$$\bar{\alpha}_t(s) \overset{\text{def}}{=} \alpha_{t-1}(s) + \alpha_{t-1}(s-1).$$

Forward pass case 2:



$$\alpha_t(s) = \begin{cases} \bar{\alpha}_t(s) y^t_{\mathbf{l}'_s} & \text{if } \mathbf{l}'_s = b \text{ or } \mathbf{l}'_{s-2} = \mathbf{l}'_s \\ \big(\bar{\alpha}_t(s) + \alpha_{t-1}(s-2)\big) y^t_{\mathbf{l}'_s} & \text{otherwise} \end{cases}$$

where

$$\bar{\alpha}_t(s) \overset{\text{def}}{=} \alpha_{t-1}(s) + \alpha_{t-1}(s-1).$$

Training time: Forward and backward

- Forward(calculate α) :

$$p(\mathbf{l}|\mathbf{x}) = \alpha_T(|\mathbf{l}'|) + \alpha_T(|\mathbf{l}'| - 1)$$

- Backward(calculate β) and combine with forward:

$$\frac{\partial p(\mathbf{l}|\mathbf{x})}{\partial y_k^t} = \frac{1}{{y_k^t}^2} \sum_{s \in lab(\mathbf{l},k)} \alpha_t(s)\beta_t(s)$$

MLE, start of model backprop

$$\frac{\partial ln(p(\mathbf{l}|\mathbf{x}))}{\partial y_k^t} = \frac{1}{p(\mathbf{l}|\mathbf{x})} \frac{\partial p(\mathbf{l}|\mathbf{x})}{\partial y_k^t}$$

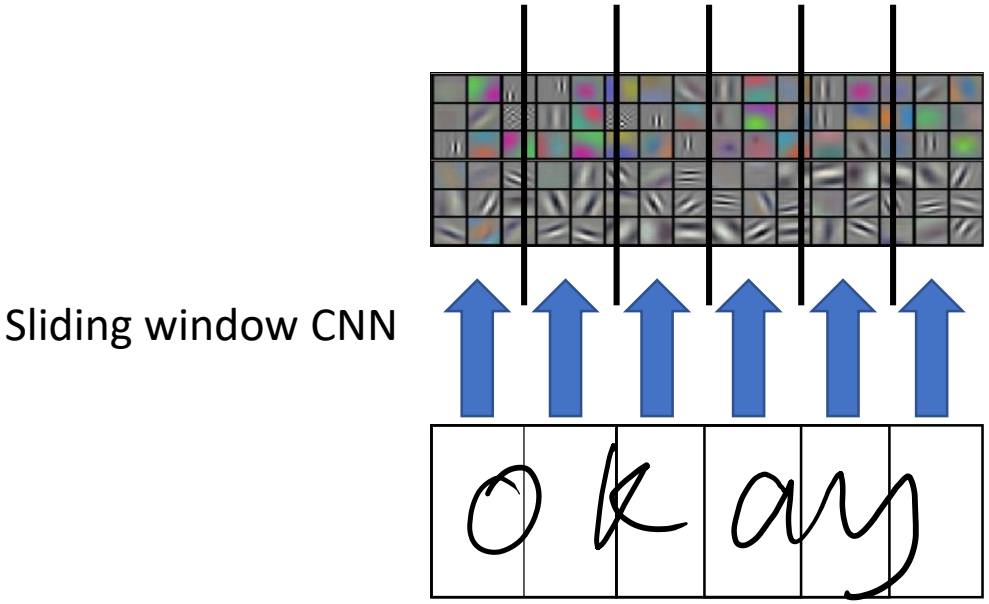
Inference strategies at test time

- Most likely alignment heuristic: $$A^* = \underset{A}{\operatorname{argmax}} \prod_{t=1}^{T} p_t(a_t \mid X)$$
- Collapsing alignments by using “blank” token as divider (Graves, 2006)
- Modified beam search and incorporating a language model (https://distill.pub/2017/ctc/)

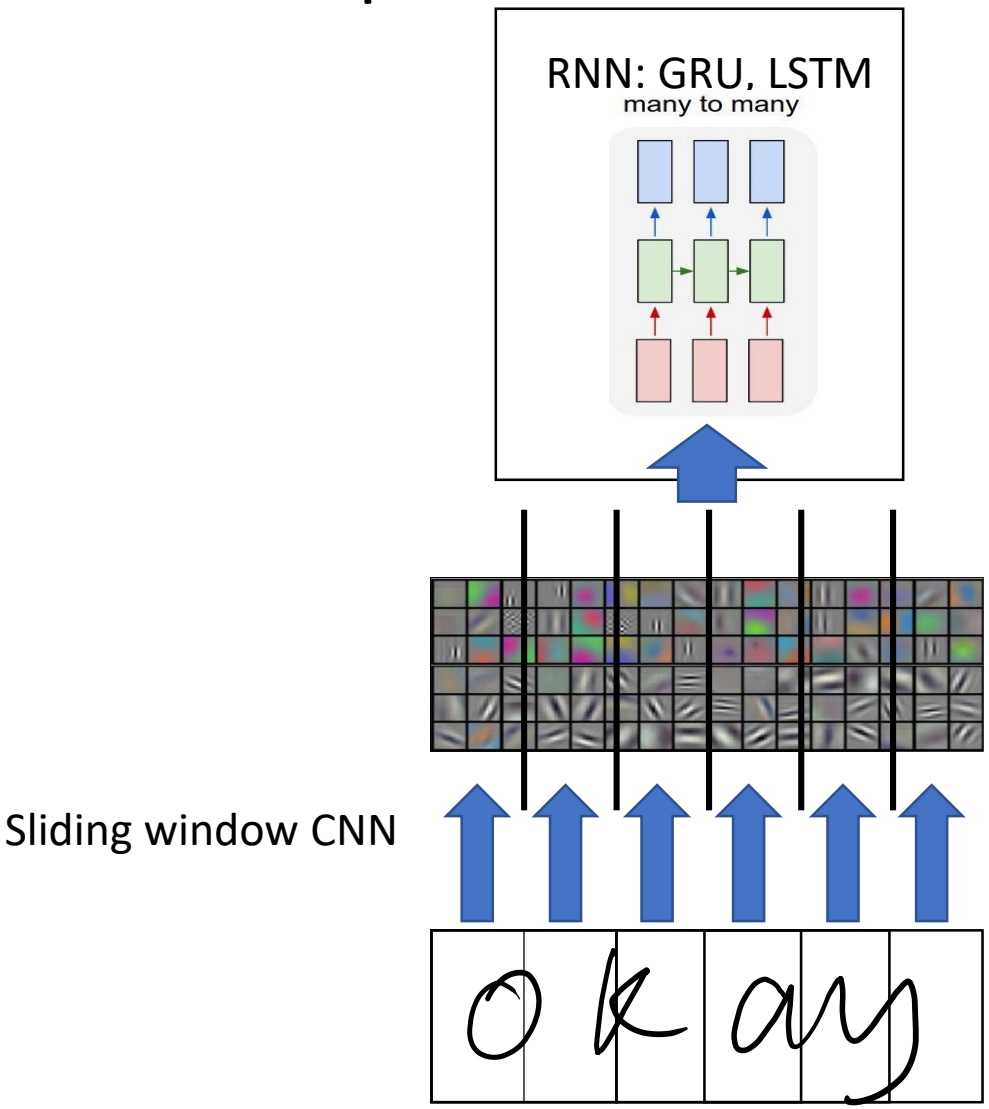
OCR w/ CTC System: Layered components

- Step 1: Visual feature extraction (CNN)
- Step 2: Sequential modeling based on visual feature sequence (RNN)
- Step 3: CTC layer to map input sequence (visual feature sequence) to output sequence (character sequence)

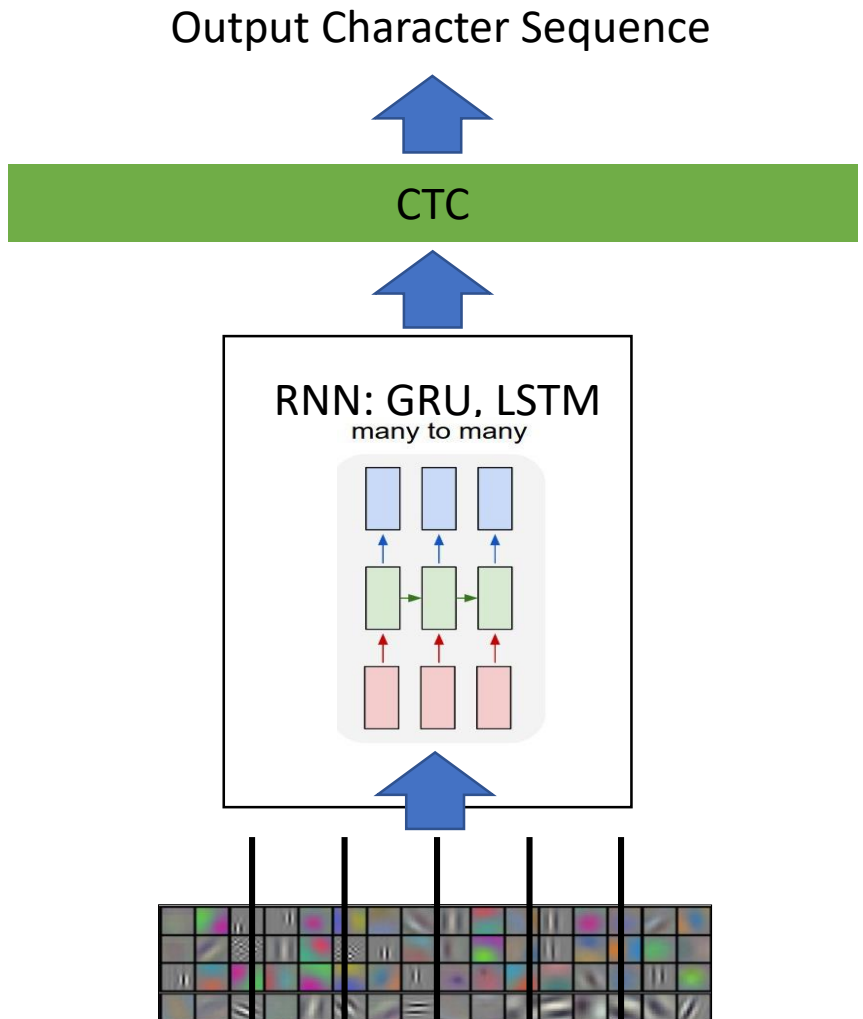
OCR w/ CTC Step 1: Visual Feature Extraction



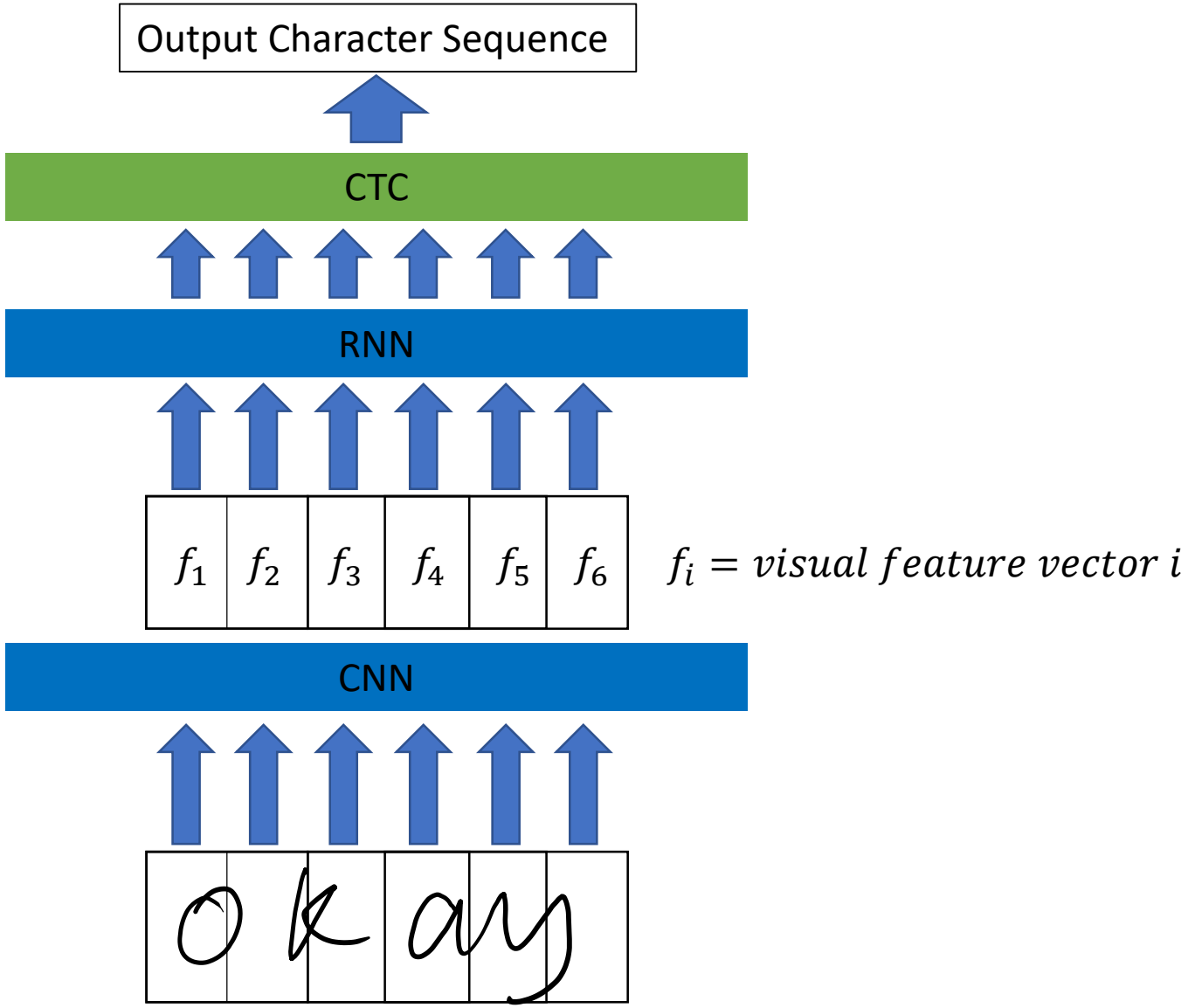
OCR w/ CTC Step 2: RNN



OCR w/ CTC Step 3: CTC Mapping



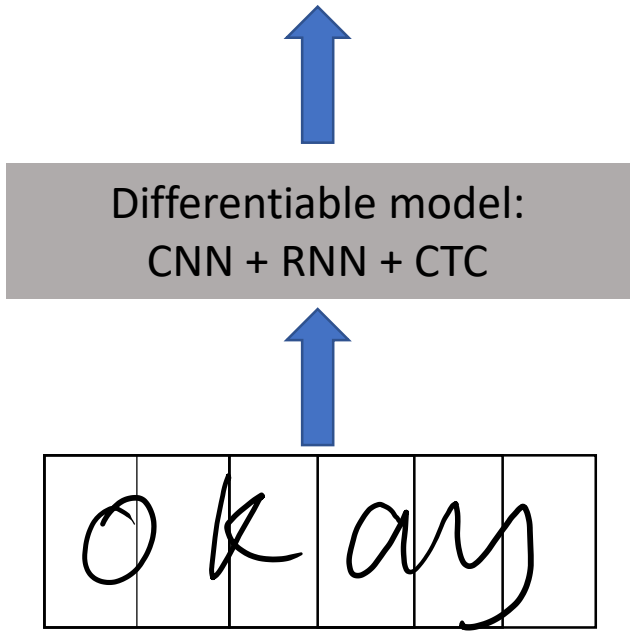
OCR w/ CTC System Overview



End-to-end trainable

https://arxiv.org/pdf/1507.05717.pdf

Output: character sequence “okay”



Input: image

Train:

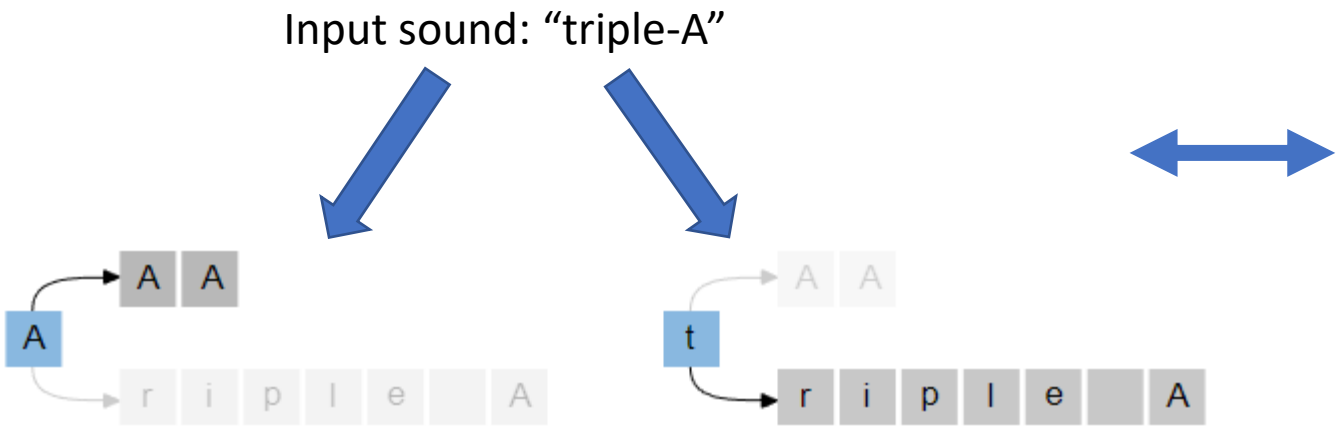
$$\operatorname*{argmax}_{\theta} P(Y|X,\theta)$$

$$= \operatorname*{argmax}_{\theta_{CNN},\theta_{RNN}} P(character\ sequence|image,\theta_{CNN},\theta_{RNN})$$

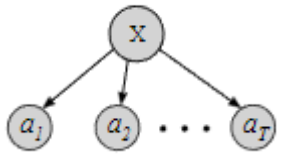
Disadvantages of CTC

https://distill.pub/2017/ctc/

- Built-in Conditional Independence: unable to learn language model



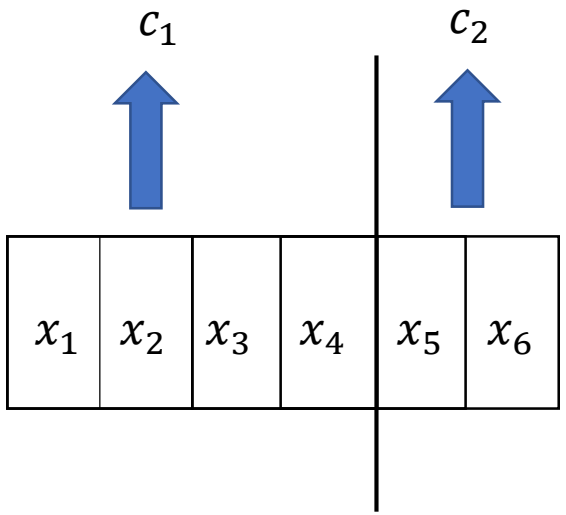
- Not explicitly expressed in CTC
- Experiments show that adding a language model boosts performance for specific settings (https://distill.pub/2017/ctc/)
- Does not learn a language model well (https://arxiv.org/pdf/1707.07413.pdf)



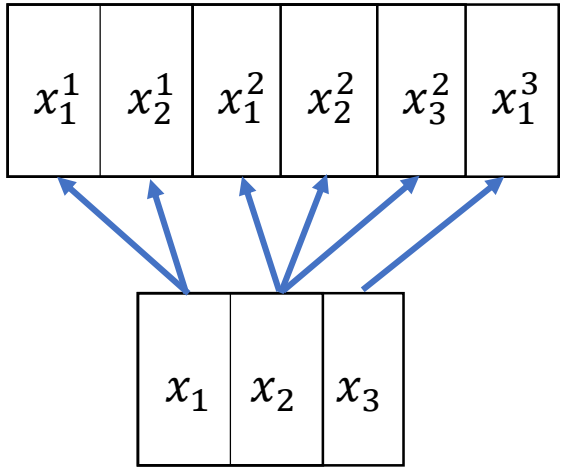
Disadvantages of CTC

https://distill.pub/2017/ctc/

- Many to one mapping (discussion question): CTC facilitates collapsing



CTC good in Many to one:
Speech recognition, OCR



CTC not so good in Many to many
(potentially expanding length of input sequence or changing order):
Machine translation, other examples?